

# Campus Ride Management Platform

Design Document | IIT Roorkee e-rickshaw ride coordination | Full-stack web application

| Item               | Summary                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem            | Campus e-rickshaw requests are often coordinated informally, causing poor visibility into demand, driver availability, and ride status.                    |
| Solution           | A responsive web platform connects passengers and drivers through authenticated dashboards, ride workflows, live updates, maps, and feedback.              |
| Stack              | React + TypeScript + Vite frontend; Java 21 Spring Boot backend; PostgreSQL; Spring Security JWT; WebSocket/STOMP; Leaflet/OpenStreetMap.                  |
| Current deployment | Local development setup with PostgreSQL. Shared hosted database is supported through backend/.env but local DB is recommended for smooth demo performance. |

## 1. Problem Understanding and Scope

The platform models a campus-scale dispatch system where passengers need quick last-mile rides and drivers need a reliable queue of requests. The system must coordinate multiple users, prevent double assignment, synchronize state across dashboards, and keep ride history available for analytics and feedback.

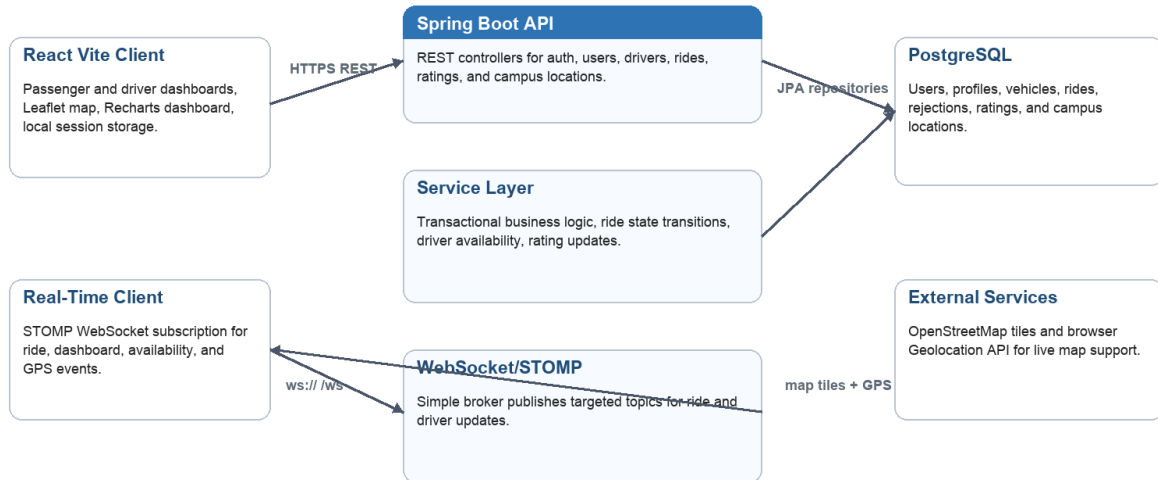
- Passengers can register, log in, manage profile details, view available drivers, request rides, track ride status, cancel active rides, and rate completed rides.
- Drivers can register with verification and vehicle data, control availability, view incoming ride requests, accept/reject rides, update ride lifecycle, share GPS location, and view dashboard statistics.
- The implementation prioritizes the mandatory ride workflow and real-time communication, then adds optional live maps and GPS tracking for stronger demonstration value.

| Requirement Area       | Implemented Coverage                                                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Authentication/profile | Passenger and driver registration/login, JWT auth, role-aware profile management, BCrypt password hashes.                         |
| Availability           | Driver OFFLINE/ONLINE/BUSY states, passenger available-driver list, WebSocket availability notifications.                         |
| Ride workflow          | Request ride, choose pickup/destination, driver accept/reject, one-driver assignment, cancel/start/complete actions.              |
| Real-time updates      | STOMP topics for new ride requests, ride status changes, driver availability, dashboard refreshes, and driver GPS changes.        |
| Dashboard/feedback     | Driver summary cards, chart, ride history, active ride panel, ratings received, passenger rating and written feedback.            |
| Optional features      | Leaflet/OpenStreetMap map, seeded IIT Roorkee campus locations, passenger/driver markers, browser Geolocation based GPS tracking. |

## 2. System Architecture

The frontend is a single-page React application. It communicates with the Spring Boot backend through REST for commands/queries and STOMP WebSockets for live events. Spring services own all business rules and transactional state transitions. PostgreSQL stores durable user, ride, rating, and location data.

## System Architecture

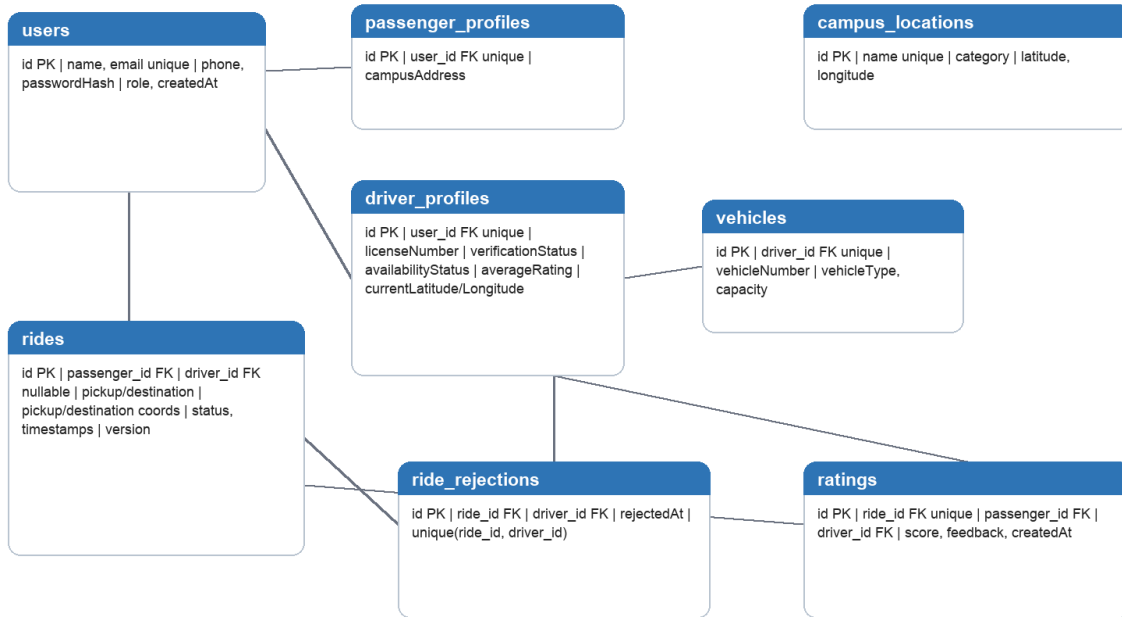


| Layer           | Responsibility                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Frontend        | Role-specific dashboards, form state, ride cards, map rendering, STOMP client subscriptions, local auth session.                  |
| API/controllers | HTTP endpoints under /api for auth, profile, drivers, rides, ratings, and campus locations.                                       |
| Services        | Validation, authorization checks, transaction boundaries, ride lifecycle rules, dashboard aggregation, realtime event publishing. |
| Database        | PostgreSQL with JPA entities and repositories; Hibernate ddl-auto update for reproducible table creation during development.      |
| Realtime        | Spring WebSocket simple broker on /ws; topics broadcast request, assignment, status, availability, dashboard, and GPS events.     |

## 3. Database Schema and ERD

The schema separates common account data from role-specific profiles. Rides reference users as passenger and optional driver, while ratings and rejections preserve ride-level history. Campus locations provide stable map coordinates for pickup and destination selection.

## Entity Relationship Diagram



Relationships: users have one passenger or driver profile; drivers have one vehicle; rides reference passenger and optional driver; ratings and rejections reference rides; campus\_

| Table              | Purpose and Key Fields                                                                                                            |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| users              | Shared account table: name, unique email, phone, passwordHash, role, createdAt.                                                   |
| passenger_profiles | One-to-one passenger extension with campusAddress.                                                                                |
| driver_profiles    | One-to-one driver extension: license, verification document/status, availability, averageRating, ratingCount, current GPS fields. |
| vehicles           | One-to-one driver vehicle record: vehicleNumber, vehicleType, capacity.                                                           |
| rides              | Ride lifecycle entity: passenger_id, driver_id, pickup/destination names and coordinates, status, timestamps, optimistic version. |
| ride_rejections    | Tracks driver rejections with unique ride_id + driver_id so rejected rides do not keep reappearing to that driver.                |
| ratings            | One rating per completed ride, linked to passenger and driver, with score and optional feedback.                                  |
| campus_locations   | Named IIT Roorkee locations with category, latitude, and longitude used by maps and ride creation.                                |

## 4. Ride Lifecycle and Consistency

| State       | Owner Action                                      | Consistency Rule                                                                                      |
|-------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| REQUESTED   | Passenger creates ride.                           | Passenger cannot create another active ride. Drivers can see request unless they already rejected it. |
| ACCEPTED    | One online driver accepts.                        | Pessimistic lock on ride row ensures only one driver can claim the ride; driver becomes BUSY.         |
| IN_PROGRESS | Assigned driver starts ride.                      | Only assigned driver can start; passenger and driver receive realtime update.                         |
| COMPLETED   | Assigned driver completes ride.                   | Driver returns ONLINE; completed ride becomes eligible for passenger rating.                          |
| CANCELLED   | Passenger or assigned driver cancels active ride. | Completed/cancelled rides cannot be cancelled again; assigned driver returns ONLINE.                  |

Concurrency is handled in the accept flow using a database pessimistic lock (findByIdForUpdate) plus status/driver checks inside a transaction. This prevents two drivers from accepting the same request under simultaneous clicks. The Ride entity also carries a version field for optimistic consistency during persistence.

## 5. Real-Time Communication Design

| Topic / Event                          | Audience          | Use                                                                                 |
|----------------------------------------|-------------------|-------------------------------------------------------------------------------------|
| /topic/rides/requests - RIDE_REQUESTED | Drivers           | New incoming ride appears without refresh.                                          |
| /topic/users/{id}/notifications        | Passenger/driver  | Ride accepted, rejected, started, completed, cancelled, or location-updated notice. |
| /topic/drivers/availability            | Passengers        | Available driver pool updates when drivers go online/offline/busy.                  |
| /topic/drivers/{id}/dashboard          | Driver            | Assigned ride and dashboard updates for that driver.                                |
| /topic/rides/{id}/driver-location      | Ride participants | Driver marker can move live while ride is accepted/in progress.                     |

Driver GPS uses the browser Geolocation API. The frontend throttles updates by time and distance, posts them to `/api/drivers/location`, and the backend publishes location events. The UI applies location events locally where possible to avoid expensive full dashboard reloads.

## 6. API Overview

| Area     | Method  | Endpoint                                       | Purpose                                                        |
|----------|---------|------------------------------------------------|----------------------------------------------------------------|
| Auth     | POST    | <code>/api/auth/register/passenger</code>      | Create passenger account and return JWT.                       |
| Auth     | POST    | <code>/api/auth/register/driver</code>         | Create driver account, vehicle, verification profile, and JWT. |
| Auth     | POST    | <code>/api/auth/login</code>                   | Authenticate existing user.                                    |
| User     | GET/PUT | <code>/api/users/me</code>                     | Read or update current profile.                                |
| Location | GET     | <code>/api/locations</code>                    | Fetch campus pickup/destination coordinates.                   |
| Driver   | POST    | <code>/api/drivers/availability/online</code>  | Set driver online or busy if already assigned.                 |
| Driver   | POST    | <code>/api/drivers/availability/offline</code> | Set driver offline when no active ride exists.                 |

|        |          |                                                     |                                                        |
|--------|----------|-----------------------------------------------------|--------------------------------------------------------|
| Driver | GET      | /api/drivers/available                              | Passenger list of online drivers.                      |
| Driver | POST     | /api/drivers/location                               | Store and broadcast driver GPS update.                 |
| Driver | GET      | /api/drivers/requests                               | Incoming ride requests excluding rejected rides.       |
| Driver | GET      | /api/drivers/dashboard                              | Stats, active ride, ride history, ratings, chart data. |
| Ride   | POST/GET | /api/rides, /api/rides/my                           | Create ride and fetch passenger/driver ride history.   |
| Ride   | POST     | /api/rides/{id}/accept reject start complete cancel | Ride assignment and lifecycle transitions.             |
| Rating | POST     | /api/ratings                                        | Rate a completed ride and update driver average.       |

## 7. Design Decisions and Scalability

| Decision                                        | Rationale                                                                                                    |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| JWT + stateless Spring Security                 | Simple reproduction, role-aware API protection, no server-side session store required.                       |
| Spring service layer transactions               | Keeps ride lifecycle validation, state mutation, and realtime publishing centralized and maintainable.       |
| Pessimistic row lock on acceptance              | Protects the strongest consistency requirement: one ride can only be assigned to one driver.                 |
| Separate ride_rejections table                  | Preserves driver-specific rejection history without mutating the ride away from REQUESTED for other drivers. |
| Campus locations as database rows               | Coordinates can be corrected through SQL/pgAdmin without changing frontend code.                             |
| Maps as optional enhancement                    | Leaflet/OpenStreetMap avoids paid API keys and gives visible passenger/driver context.                       |
| Local PostgreSQL for demo, env-driven DB config | Local DB gives smooth demos; .env can switch to shared hosted DB for team testing.                           |

- Scalability path: add pagination for ride history, indexes on ride status/passenger/driver/requestedAt, and replace the simple broker with a message broker such as RabbitMQ for multi-instance WebSocket scaling.
- Operational path: use environment variables for secrets, stricter production CORS, HTTPS, managed PostgreSQL, and database migrations through Flyway or Liquibase.
- Future bonus scope: ride scheduling, simulated UPI/QR payments, demand analytics, demand forecasting, and richer heatmap/dashboard insights.

## 8. Implemented Feature Checklist

| Category           | Implemented Features                                                                                                                                                                                                  |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mandatory          | Authentication, passenger/driver registration, profiles, vehicle/verification data, availability, ride request, accept/reject, single assignment, ride states, WebSocket updates, driver dashboard, ratings/feedback. |
| Optional completed | Live maps, campus location coordinates, pickup/destination markers, assigned driver marker, driver GPS tracking, live driver marker updates.                                                                          |
| Not included yet   | Ride scheduling, digital payments, demand analytics, demand forecasting, deployment production hardening.                                                                                                             |